Smart Home Control System: ROS2 Capstone Project (Simulation-Based with Console Output)

Project Goal

Design and implement a simulated smart home control system using ROS2, focusing on software development and ROS2 communication mechanisms (topics, services, and actions). No physical sensors or actuators will be used in this project. Crucially, all nodes must provide informative console output during runtime to illustrate their activity and logic.

Project Description

Imagine a home where technology seamlessly integrates to enhance comfort, convenience, and security. In this project, you will build a ROS2-based system to simulate the control of essential aspects of a smart home environment, including lighting, temperature regulation, and a basic security system.

You will develop a network of ROS2 nodes that communicate with each other to monitor simulated sensor data, process information, and control simulated actuators. This will involve implementing publishers, subscribers, service servers and clients, and action servers and clients. All sensor data will be generated randomly within the respective nodes using code.

To aid in debugging and understanding the system's behavior, each node must print illustrative information to the console during runtime. This output should clearly indicate:

- The node's current state and actions.
- The values of simulated sensor readings.
- The logic behind decisions and control actions.
- Any relevant events or changes in the system.

Core Features

1. **Lighting Control:**
   - Automatic Brightness Adjustment: Simulate an ambient light sensor by generating random light level readings within your light_control_node. Use this data to automatically adjust the brightness of simulated smart bulbs. Print the simulated light level to the console, along with the corresponding brightness adjustment made.
   - Manual Control: Allow users to turn simulated lights ON, OFF, or set them to a specific dimming level. Print messages to the console indicating the received commands and the resulting light state.

2. **Temperature Control:**
   - Temperature Monitoring: Generate random temperature readings within your temperature_control_node to simulate a temperature sensor. Print the simulated temperature readings to the console.
   - Climate Control: Implement a simulated thermostat to regulate a simulated heater/cooler, maintaining a desired temperature based on the simulated temperature readings. Print

messages indicating the current temperature, the target temperature, and whether the heater/cooler is activated.

3. **Security System:**
   - Motion Detection: Simulate a PIR motion sensor by randomly generating "motion detected" events within your security_node. Print messages to the console whenever motion is detected or not detected.
   - Alarm System: Trigger a simulated alarm (print messages to the console, e.g., "ALARM TRIGGERED!") when simulated motion is detected while the system is armed.
   - Arm/Disarm Functionality: Allow users to arm or disarm the simulated security system. Print messages to the console indicating the current security state (ARMED/DISARMED).

ROS2 Implementation

Nodes:
- light_control_node: Manages the state and control of the simulated lights.
- temperature_control_node: Regulates the simulated temperature.
- security_node: Monitors simulated sensors and triggers the simulated alarm.
- user_interface_node: Provides a command-line or basic GUI for user interaction.

Topics:
- /ambient_light: (std_msgs/Float32) Publishes simulated readings from the ambient light sensor.
- /temperature: (std_msgs/Float32) Publishes simulated temperature readings.
- /motion_detected: (std_msgs/Bool) Publishes True when simulated motion is detected.
- /light_state: (std_msgs/String) ("ON", "OFF", "DIMMED") Indicates the current state of the simulated lights.
- /security_state: (std_msgs/String) ("ARMED", "DISARMED", "TRIGGERED")

Services:
- /light_control: (Service type with request/response fields for "ON", "OFF", "DIM")
- /set_temperature: (Service type with a request field for the desired temperature)
- /arm_security: (Empty service to arm the security system)
- /disarm_security: (Empty service to disarm the security system)

Actions:
- /dim_lights: (Action type with goal: target brightness level, feedback: current brightness, result: success/failure)

Deliverables

- Working ROS2 System: A functional smart home control system with all specified features implemented.
- Well-documented Code: Clear and concise code with comments explaining the logic and functionality.
- Project Report: A document describing the system architecture, ROS2 communication

mechanisms, and implementation details.
- Presentation: A presentation summarizing the project, including demonstrations of the system's capabilities.

Assessment

Your project will be evaluated based on:
- Functionality: Correct implementation of all required features.
- ROS2 Concepts: Effective use of topics, services, and actions for inter-node communication.
- Code Quality: Well-structured, readable, and documented code.
- Console Output: Clarity and usefulness of the console output for debugging and understanding the system.
- Project Report: Clarity and completeness of the project documentation.
- Presentation / Video: Effective communication of project goals, implementation, and results.